

Universidad del Valle de Guatemala

Programación Orientada a Objetos

Sección 40

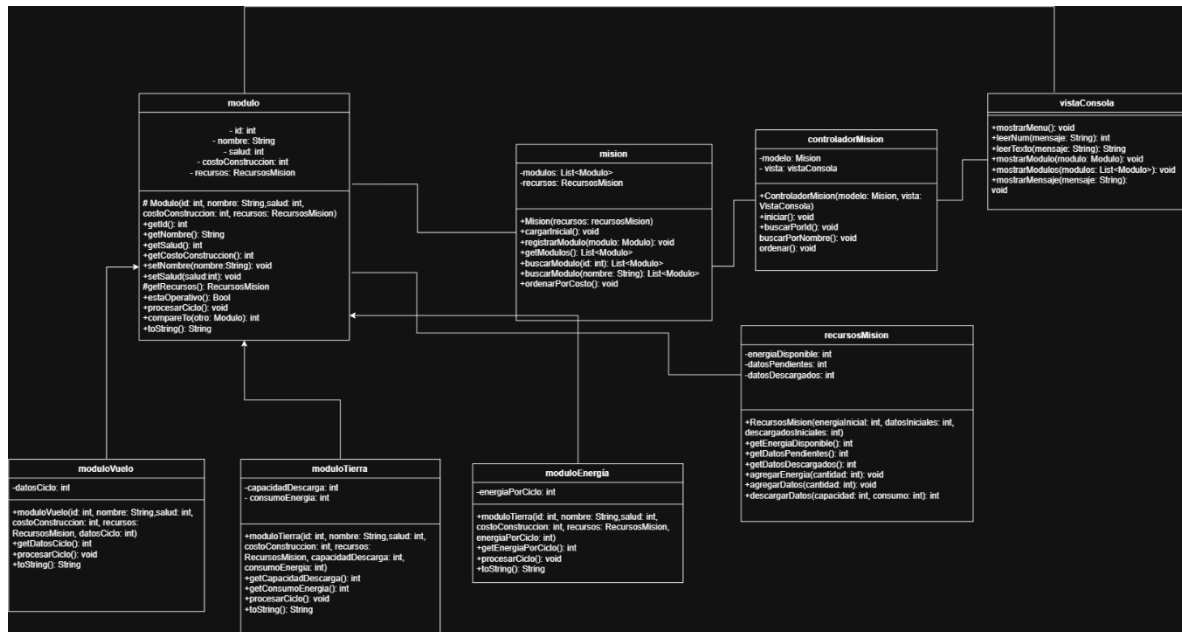
Catedrático: Santiago Solorzano

Ejercicio Polimorfismo y Herencia

Juan Miguel Cordon – 26342

Luis Diego Nuñez – 26321

Diagrama UML



Análisis del problema:

Problema y Objetivo: Se debe crear un juego donde el operador de la misión debe consultar los componentes que se mantienen activos en el Quetzal-2. El sistema debe representar distintos módulos de vuelo, de tierra y energía que permita definir los distintos comportamientos de estos módulos y como es que estos se relacionan entre sí.

Clases Modelo

- Modulo
 - Propósito
 - Es una clase que representa las características de los módulos de la misión, es una clase padre de los módulos de vuelo, tierra y energía.
 - Atributos
 - id: Identifica el módulo dentro de la misión con un número único.
 - nombre: Almacena el nombre del módulo.

- salud: representa la salud del módulo con un número entre el 0 al 100.
 - costoConstruccion: Registra el costo de la construcción del módulo.
 - recursos: Son los recursos compartidos de la misión.
- Métodos:
 - Modulo(id: int, nombre: String, salud: int, costoConstruccion: int, recursos: RecursosMision): Inicia y valida los atributos comunes por un constructor usado por las clases hijas
 - estaOperativo(): boolean: Determina si el módilo puede funcionar, es decir cuando la salud es > 0
 - procesarCiclo(): void: Crea la operación que cada clase hija debe implementar para responder a un ciclo
 - compareTo(otro: Modulo): int: Compara el costo del módulo con el del otro para no confundirlos. Si son iguales, compara los IDs.
 - toString(): String: Redefine el método de Object para devolver el tipo específico, id, nombre, salud, etc.
 - getId(): int: getter del ID
 - getNombre(): String: getter de Nombre
 - getSalud(): int: getter Salud
 - getCostoConstruccion(): int: getter de Costo Salud
 - getRecursos(): RecursosMision: getter de recursos
 - setNombre(nombre: String): void: Modifica el nombre
 - setSalud(salud: int): void: Modifica la salud y verifica que esté entre 0 y 100
- ModuloVuelo
 - Propósito
 - Es el módulo que recolecta los datos de vuelo en el simulador.
 - Atributos
 - datosPorCiclo: Almacena la capacidad de datos recolectada en un ciclo
 - Métodos
 - ModuloVuelo(id: int, nombre: String, salud: int, costo: int, recursos: RecursosMision, datosPorCiclo: int): Crea el módulo e inicia la recolección
 - procesarCiclo(): String: Redefine el método de Modulo y agrega datosPorCiclo a los datos pendientes de la misión
 - toString(): String: Redefine el método para incluir la información y la cantidad de datos recolectados por ciclo
 - getDatosPorCiclo(): int: Consulta la capacidad de recolección
- ModuloTierra
 - Propósito

- Representa la antena que descarga la información del satélite usando energía
 - Atributos
 - capacidadDescarga: Registra la capacidad máxima de los datos que puede descargar por ciclo
 - consumoEnergia: Registrar la cantidad de energía que se necesita para hacer una descarga
 - Métodos
 - ModuloTierra(id: int, nombre: String, salud: int, costo: int, recursos: RecursosMision, capacidadDescarga: int, consumoEnergia: int): Crea el módulo con sus características de descarga
 - procesoCiclo(): void: Redefine el método de Modulo para que si está operativo, solicita a los recursos de la misión la descarga de datos
 - toString(): String: Redefine el método para incluir la capacidad de descarga y el consumo de energía
 - getCapacidadDescarga(): int: getter de CapacidadDescarga
 - getConsumoEnergia(): int: Getter de ConsumoEnergia
- ModuloEnergia
 - Propósito:
 - Sirve para generar energía para la misión. Hereda de Modulo
 - Atributos
 - energiaPorCiclo: Almacena la cantidad de energía que se genera en un ciclo
 - Métodos
 - ModuloEnergia(id: int, nombre: String, salud: int, costo: int, recursos: RecursosMision, energiaPorCiclo: int): Crea el módulo con la capacidad de generar energía
 - procesarCiclo(): void: Redefine el método para agregar su producción a energía disponible
 - ToString(): String: Redefine el método para incluir los datos de producción de energía
 - getEnergiaPorCiclo(): int: Getter de EnergiaPorCiclo
- RecursosMision
 - Proposito:
 - Administra la energía, los datos pendientes y los datos descargados. Mantiene las cantidades consistentes usadas por los módulos.
 - Atributos
 - energiaDisponible: Registra la energía disponible para hacer operaciones
 - datosPendientes: Registra los datos recolectados que aún no se descargan

- datosDescargados: Registra la cantidad de datos descargados
- Métodos
 - RecursosMision(energiaInicial: int, datosIniciales: int, descargadosIniciales: int): Crea los recursos para la misión
 - agregarEnergia(cantidad:int): void: Aumenta la energía disponible
 - agregarDatos(cantidad:int): void: Aumenta los datos pendientes
 - descargarDatos(capacidad: int, consumo: int): int: Verifica que existen datos y energía suficiente para descargar datos entre la capacidad máxima y la cantidad de datos pendientes, descontar energía y aumentar los datos descargados
 - getEnergiaDisponible(): int: Getter de EnergiaDisponible, consulta la energía disponible
 - getDatosPendientes(): int: Getter de DatosPendientes, consulta los datos pendientes
 - getDatosDescargados(): int: Getter de DatosDescargados, consulta los datos descargados
- Mision
 - Proposito:
 - Administra los módulos y da la carga inicial, ordenamiento, consulta y búsqueda de las operaciones
 - Atributos:
 - modulos: List<Modulo>: Almacena los tres tipos de módulos en una lista
 - recursos: Mantiene los recursos compartidos de la misión
 - Métodos:
 - Mision(recursos: RecursosMision): Crea una misión para asignarle sus recursos
 - cargaInicial(): void: Crea y registra 10 modulos de los 3 tipos y se ejecuta una sola vez sobre una misión
 - registrarModulo(modulo: Modulo): void: Incorpora un módulo, verifica que su ID sea único y que use los recursos de la misión
 - getModulos(): List<Modulo>: Es un getter de la lista de módulos para poder consultarse
 - buscarModulo(id: int): Optional<Modulo>: Busca por el ID y devuelve el módulo encontrado y si no se encuentra, un resultado vacío
 - buscarModulo(nombre: String): List<Modulo>: Devuelve todos los módulos con el mismo nombre
 - ordenarPorCosto(): void: Ordena la lista por costo ascendente usando el orden de compareTo()

Interfaces

- Comparable<Modulo>
 - Propósito:
 - Establece la comparación para ordenar todos los módulos
 - Metodos:
 - compareTo(otro: Modulo): int: Establece la comparación, debe devolver un valor negativo, cero o positivo según el orden por costo y por el ID

Vista

- Vista Consola
 - Propósito:
 - Presenta el menú del programa, solicita entradas al operador y muestra los resultados del controlador
 - Métodos:
 - mostrarMenu(): void: Muestra las opciones de listar, buscar por ID, por nombre, ordenar por costo y salir
 - leerEntero(mensaje: String): int: Solicita un int y repite la lectura si la entrada no tiene el formato correcto
 - leerTexto(mensaje: String): String: Solicita y devuelve el texto ingresado
 - mostrarModulo(modulo: Modulo): void: Presenta la información del módulo por su implementación de toString()
 - mostrarModulos(módulos: List<Modulo>): void: Recorre los módulos y los muestra
 - mostrarMensaje(mensaje: String): void: Presenta avisos, errores de entrada o cuando no hay resultados en una búsqueda

Controlador

- ControladorMision
 - Propósito
 - Coordinar las acciones del operador y conecta la vista con Mision
 - Atributos
 - modelo: Mision: Mantiene la referencia al modelo que administra el catalogo
 - vista: VistaConsola: Mantiene la referencia a la vista para usar la vista con el operador
 - Métodos:
 - ControladorMision(modelo: Mision, vista: VistaConsola): Crea el controlador

- `iniciar(): void`: Solicita la carga inicial y mantiene el menú hasta que se elija “salir”
- `listar(): void`: Solicita el catálogo al modelo y lo envía a la vista
- `buscarPorId(): void`: Solicita un identificador, busca el módulo y lo muestra
- `buscarPorNombre(): void`: Solicita un nombre, hace la búsqueda y presenta cual coincide
- `ordenar(): void`: Solicita el ordenamiento al modelo y presenta el catálogo ordenado.

Clase Main

- Main
 - Propósito:
 - Sirve como el main del programa y crea los objetos para iniciar el programa
 - Métodos:
 - `Main(args: String[]): void`: Crea `RecursosMision`, `Mision`, `VistaConsola` y `ControladorMision`. Luego inicia en el controlador